

```
GNU nano 7.2 virtual_address.c
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

```
root@unknown:~# nano virtual_address.c
root@unknown:~# gcc virtual_address.c -o virtual_address
root@unknown:~# ./virtual_address
Value of x = 10
Virtual address of x = 0x7ffdf9b4d4d4
root@unknown:~# nano virtual_address.c
root@unknown:~#
```

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address    : %p\n", (void *)main);
    printf("Data address     : %p\n", (void *)&global);
    printf("BSS address        : %p\n", (void *)&bss_variable);
    printf("Heap address        : %p\n", (void *)heap);
    printf("Stack address       : %p\n", (void *)&local);

    free(heap);

    return 0;
}_
```

```
root@unknown:~# nano memory_segments.c
root@unknown:~# gcc memory_segments.c -o memory_segments
root@unknown:~# ./memory_segments
Code address    : 0x566cc0c221a9
Data address     : 0x566cc0c25010
BSS address      : 0x566cc0c25018
Heap address     : 0x566cf2fb62a0
Stack address    : 0x7fffe7d95acc
root@unknown:~# _
```

root@unknown: ~

× + ▾

GNU nano 7.2

```
#include <stdio.h>
int main() {
    int page_table[4] = {1, -1, 3, -1};
    int page;
    printf("Enter page number (0-3): ");
    scanf("%d", &page);
    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }
    return 0;
}
```

Stack address : 0x7fffe7d73dc0

root@unknown:~# nano page_table.c

root@unknown:~# nano page_table.c

root@unknown:~# gcc page_table.c -o page_table

root@unknown:~# ./page_table

Enter page number (0-3): 2

Page is already in memory.

Physical frame = 3

root@unknown:~#

root@unknown: ~

×

+

▽

GNU nano 7.2

```
#include <stdio.h>
int main() {
    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;
    int virtual_address = 250;
    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;
    int frame = page_table[page];
    int physical_address = frame * page_size + offset;
    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number      = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);
    return 0;
}
```

```
root@unknown:~# nano address_translation.c
root@unknown:~# gcc address_translation.c -o address_translation
root@unknown:~# ./address_translation
Virtual Address = 250
Page Number      = 2
Offset           = 50
Frame Number      = 1
Physical Address = 150
root@unknown:~#
```

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p;
    p = malloc(5 * sizeof(int));
    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }
    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }
    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }
    printf("\n");
    free(p);
    return 0;
}
```

```
root@unknown:~# nano dynamic_memory.c
root@unknown:~# gcc dynamic_memory.c -o dynamic_memory
root@unknown:~# ./dynamic_memory
10 20 30 40 50
root@unknown:~#
```

root@unknown: ~

×

+

∨

GNU nano 7.2

```
#include <stdio.h>
int main() {
    int *p = NULL;
    printf("Before accessing memory\n");
    *p = 100;
    printf("After accessing memory\n");
    return 0;
}
```

```
root@unknown:~# nano null_pointer.c
root@unknown:~# gcc null_pointer.c -o null_pointer
root@unknown:~# ./null_pointer
Before accessing memory
Segmentation fault (core dumped)
root@unknown:~#
```